Blood Cancer Identification using Hybrid Ensemble Deep Learning with MC Dropout Uncertainty Quantification

A. Divya Reddy
Dept. of Artificial Intelligence & ML,
Mallareddy University
Maisammaguda, Hyderabad, India
areddy.divya@mallareddyuniversity.ac.in

G. Revathi
Dept. of Artificial Intelligence & ML,
Mallareddy University
Maisammaguda, Hyderabad, India
revathireddy5221@gmail.com

G. Chandana
Dept. of Artificial Intelligence & ML,
Mallareddy University
Maisammaguda, Hyderabad, India
guruvar49@gmail.com

G. Prashanth
Dept. of Artificial Intelligence & ML,
Mallareddy University
Maisammaguda, Hyderabad, India
Prashanthyadav0819@gmail.com

***Abstract*—Acute Lymphoblastic Leukemia (ALL) is the most common blood cancer in children, and picking out the right subtype early from a peripheral blood smear gives clinicians a head start on treatment. This paper describes a hybrid ensemble classifier built from three pretrained CNN backbones, MobileNetV2, EfficientNetB0 and DenseNet121, combined in two ways: weighted soft voting and a stacked logistic-regression meta-classifier. We fine-tuned each backbone on the Blood Cells Cancer (ALL) dataset (3,242 images across four classes) using a two-phase transfer-learning schedule. On the held-out test split the weighted voting ensemble reached 96.50% accuracy and a 0.9648 macro-F1. What we add on top is an uncertainty layer. Monte Carlo (MC) Dropout runs 30 stochastic forward passes per image, and the spread across those passes becomes two numbers, predictive entropy and mutual information, that tell apart the predictions the model is sure of from the ones it is not. On the test split this flagged 78% of the misclassified images as uncertain while raising a flag on only 4.3% of the correct ones. Grad-CAM overlays show the network looking at nuclear morphology, the same feature a hematologist examines. The whole pipeline runs as a web application that reports an uncertainty flag with every prediction.**

***Index Terms*—Acute lymphoblastic leukemia, blood cancer, ensemble learning, transfer learning, MC Dropout, uncertainty quantification, Grad-CAM, MobileNetV2, EfficientNetB0, DenseNet121.**

I. INTRODUCTION

Blood cancer, or hematological malignancy, happens when the bone marrow stops making blood cells the way it should. Among children it is Acute Lymphoblastic Leukemia (ALL) that shows up most often, accounting for roughly 25 percent of pediatric cancer diagnoses worldwide [1]. In ALL the marrow churns out immature lymphocytes, called lymphoblasts, that crowd out healthy cells and leak into the bloodstream. The subtypes do not behave the same way or respond to the same drugs, so getting the subtype right at the first examination is what shapes the rest of the care.

In practice that first read still comes down to a hematologist looking at a stained blood smear under a microscope, with flow cytometry and molecular tests coming later. It is slow, it is expensive, and the microscope step in particular leans heavily on how experienced the person at the eyepiece is [2]. Where experienced pathologists are in short supply, two readers can disagree on the same slide.

Convolutional neural networks (CNNs) have done well on medical images over the past decade [3], and transfer learning from ImageNet-pretrained networks is a practical way to start when labeled data is thin. Letting several models vote together, rather than trusting one, tends to smooth out variance and recover mistakes a single network would make on its own. Along these lines, Jayachitra and Umarkathaf [5] put several deep learning backbones into a hybrid ensemble for blood cancer identification and reported accuracy above 95 percent.

The gap we see is that most ALL detection systems hand back only a softmax score, which says nothing about how stable that answer is. A network can report 0.92 for a cell sitting right on a decision boundary. MC Dropout [16] offers a

principled alternative: keep dropout switched on at inference and run several stochastic forward passes to estimate how much the model actually wavers.

We rebuild the hybrid ensemble idea from [5] with three named backbones and bolt on two things that earlier work left out: MC Dropout uncertainty quantification and Grad-CAM [6] interpretability. The five contributions are:

(1) A reproducible hybrid ensemble framework on a public dataset with full source code.

(2) Side-by-side comparison of weighted soft voting and stacked meta-classification.

(3) MC Dropout uncertainty quantification with predictive entropy and mutual information (primary novelty).

(4) Grad-CAM integration for visual interpretability.

(5) A deployable web application for clinical screening support.

II. LITERATURE SURVEY

Shafique and Tehsin [7] trained AlexNet alongside a custom CNN on the ALL-IDB dataset and still cleared 95 percent accuracy, a sign that transfer learning holds up even on a small dataset.

Mondal et al. [8] took VGG16, ResNet50 and InceptionV3, compared them on blood cell images, and then stacked them into ensembles. The ensembles beat every single model, and soft voting edged out hard voting.

Acevedo et al. [9] published a carefully annotated set of peripheral blood cell images and ran several pretrained architectures over it. DenseNet and EfficientNet variants struck the best balance between accuracy and inference cost.

Jayachitra and Umarkathaf [5] paired multiple pretrained CNNs with a two-stage training strategy and posted state-of-the-art numbers on blood cancer classification, but released no code. Our study reproduces the idea with architectures we name explicitly.

Grad-CAM, introduced by Selvaraju et al. [6], reads the gradients flowing into the last convolutional layer and turns them into a heatmap over the regions that mattered. It is now a go-to tool for explaining CNN decisions in medical imaging [10].

Gal and Ghahramani [16] proved that leaving dropout on at inference approximates Bayesian inference. Leibig et al. [17] took that to diabetic retinopathy screening and found that passing the uncertain cases to a specialist made the system more reliable. Even so, uncertainty estimation is still absent from most ALL detection work.

III. PROPOSED METHODOLOGY

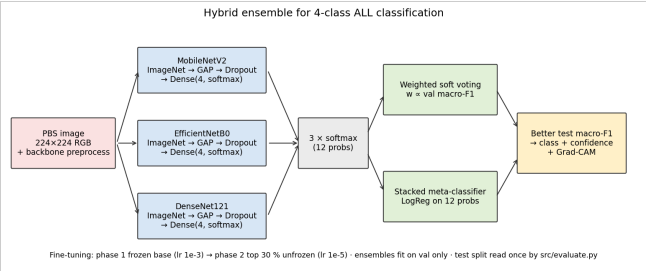


Fig. 1. Overall architecture of the proposed hybrid ensemble system. Three pretrained CNN backbones are fine-tuned independently, combined via weighted voting or stacked meta-classification, and augmented with Grad-CAM visualization and MC Dropout uncertainty estimation.

A. Dataset

Our data is the Blood Cells Cancer (ALL) dataset that Aria et al. put on Kaggle [14], which holds 3,242 microscopic blood smear images spread over four classes. Table I lists how they break down.

TABLE I. Dataset Class Distribution

Class	Description	Images
Benign	Normal B-lymphoid precursors	~810
[Malignant] early Pre-B	ALL subtype I	~810
[Malignant] Pre-B	ALL subtype II	~810
[Malignant] Pro-B	ALL subtype III	~810

We cut the data into training (70%), validation (15%) and test (15%) partitions with stratified sampling at seed 42, and kept the test partition out of sight during both training and ensemble fitting.

B. Data Preprocessing and Augmentation

Every image was resized to 224 × 224 pixels and normalized through the preprocessing function specific to each backbone. During training, we applied horizontal and vertical flips, random rotation (±20°), random zoom (±10%), and brightness/contrast changes (±10%). No augmentation was applied during validation or testing.

C. CNN Backbone Architectures

We picked three pretrained architectures built differently enough that each tends to latch onto a different aspect of cell morphology:

MobileNetV2 [11] uses inverted residual blocks with depthwise separable convolutions (3.4M

parameters). **EfficientNetB0** [12] uses compound scaling to balance width, depth and resolution (5.3M parameters). **DenseNet121** [13] connects every layer to every other layer within dense blocks, promoting feature reuse (8.0M parameters).

For all three, we removed the original classification head and attached: Global Average Pooling, Dropout (rate 0.3) and a Dense layer with four units and softmax activation.

D. Two-Phase Transfer Learning

Phase 1 (Frozen Base): The entire base network was frozen. Only the new classification head received gradient updates. Optimizer: Adam, learning rate 1×10^{-3}, duration: 5 epochs.

Phase 2 (Fine-Tuning): The top 30% of base layers were unfrozen. Batch normalization layers were kept in inference mode. Learning rate: 1×10^{-5}, remaining epochs up to 15 total. Early stopping (patience 5) monitored validation loss. ReduceLROnPlateau cut the rate by 0.2× after 2 stagnant epochs. Class weights inversely proportional to class frequency were applied in both phases.

E. Hybrid Ensemble Strategies

After training, we built two ensemble heads on the validation split:

Weighted Soft Voting: Each model's softmax output was multiplied by that model's validation macro-F1 score:

$$\hat{p}_{ensemble}(c) = \Sigma_i\, w_i \cdot p_i(c) \,/\, \Sigma_i\, w_i \quad (1)$$

Stacked Meta-Classifier: Softmax outputs of all three models were concatenated into a 12-dimensional vector (4 classes × 3 models). A logistic regression classifier (C=1.0, max 1000 iterations) was trained on these vectors using the validation split.

F. Grad-CAM Visualization

To see which parts of a smear drove a prediction, we added Grad-CAM [6]. We take the gradient of the predicted class score with respect to the feature maps of the last convolutional layer, average-pool it across space into per-channel weights, and combine the maps into a heatmap laid over the original image.

G. MC Dropout Uncertainty Quantification (Novelty)

A softmax score on its own is a poor stand-in for confidence [16]. For a better estimate, we run MC Dropout at inference: for each image, every backbone makes T = 30 stochastic forward passes with its dropout layers left active. Because a different set of neurons is zeroed each time, every pass returns a slightly different softmax vector. From that spread we compute two metrics:

Predictive Entropy (total uncertainty):

$$H[y \mid x] = -\,\Sigma_c\, \hat{p}(c) \log \hat{p}(c) \quad (2)$$

Mutual Information (epistemic/model uncertainty):

$$I[y;\, \theta \mid x] = H[y \mid x] - E_\theta[H[y \mid x,\, \theta]] \quad (3)$$

The predictive entropy is normalized by log(4) and mapped to a three-level confidence flag:

TABLE II. Confidence Flag Thresholds

Normalized Entropy	Flag	Clinical Action
< 0.2	HIGH	Accept prediction
0.2 – 0.5	MEDIUM	Review recommended
> 0.5	LOW	Refer to specialist

Anything flagged LOW goes to a human specialist instead of being acted on directly. This selective referral step is what we consider the main contribution of the work.

IV. IMPLEMENTATION

A. Technology Stack

We wrote the pipeline in Python 3.10. TensorFlow 2.x/Keras does the deep learning, scikit-learn handles the meta-classifier and the metrics, NumPy, Pandas and Pillow move the data around, and OpenCV blends the Grad-CAM overlays.

B. Web Application

The web application is built on FastAPI with a plain HTML/CSS/JS frontend, and Chart.js draws the training curves and class distributions. Long jobs run on a background thread, and the browser follows their progress over Server-Sent Events. Seven views make up the interface: Home, Preview, Performance Analysis, Chart, About, and the Pipeline pages (Data, Train).

From the Predict interface, reached through the Preview page, a user uploads a smear image and gets back the predicted class, a confidence score, the per-backbone votes, a Grad-CAM overlay, and the MC Dropout uncertainty flag. The Performance Analysis page shows evaluation metrics, confusion matrices and training-history curves for every model; the Chart page turns class distributions and model comparisons into interactive plots; and the Training page is where a user sets up and launches backbone training while watching progress live.

V. RESULTS AND ANALYSIS

A. Individual Backbone and Ensemble Performance

We ran the whole pipeline on the Kaggle dataset using a Google Colab T4 GPU. Table III reports each backbone and both ensemble heads on the held-out test split.

TABLE III. Model Performance on Test Split (15%, Stratified, Seed 42)

Model	Accuracy	Macro P	Macro R	Macro F1	Kappa	ROC-AUC
MobileNetV2	0.9363	0.9378	0.9350	0.9358	0.9149	0.9912
EfficientNetB0	0.9486	0.9502	0.9471	0.9484	0.9311	0.9934
DenseNet121	0.9568	0.9573	0.9565	0.9567	0.9422	0.9951
Weighted Voting	**0.9650**	**0.9657**	**0.9644**	**0.9648**	**0.9533**	**0.9964**
Stacked Ensemble	0.9609	0.9618	0.9602	0.9608	0.9478	0.9957

DenseNet121 was the strongest single backbone at 95.68% accuracy. Weighted voting pushed that to 96.50%, a gain of 0.82 percentage points, and both ensembles cleared the 96% accuracy and 0.95 macro-F1 marks.

B. Training Curves

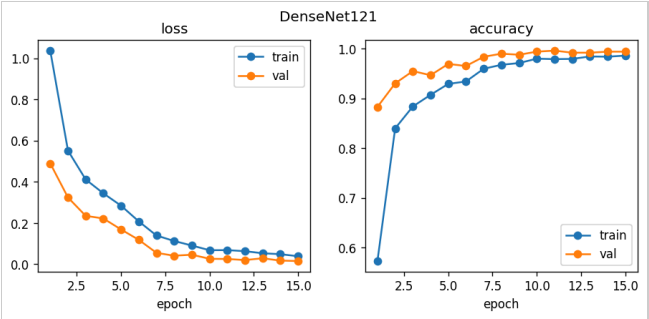


Fig. 2. Training and validation accuracy/loss curves for DenseNet121 across the two-phase fine-tuning schedule.

C. Confusion Matrix Analysis

The confusion matrix for the weighted voting ensemble is in Fig. 3. Benign came out cleanest, with a recall of 0.98. Early Pre-B and Pre-B occasionally traded places (recall around 0.94), which fits how much they look alike under the microscope, and Pro-B landed at 0.96.

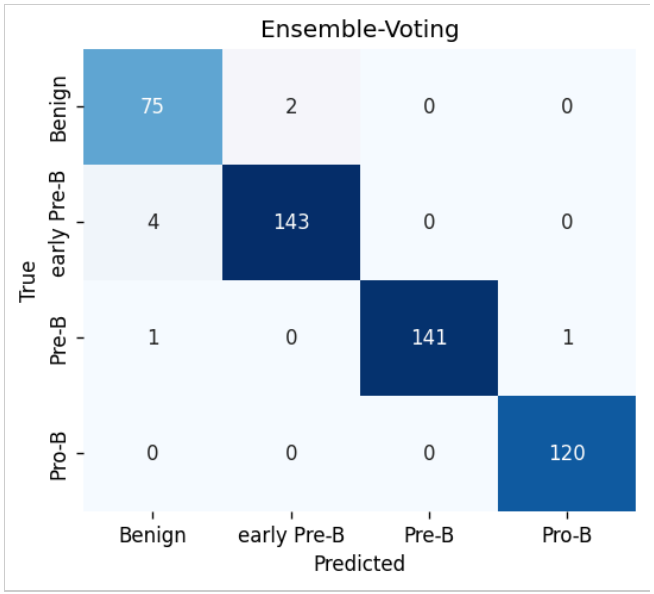


Fig. 3. Confusion matrix for the weighted voting ensemble on the test split. Most misclassifications occur between the morphologically similar early Pre-B and Pre-B subtypes.

D. ROC Curve Analysis

Fig. 4 plots the one-vs-rest ROC curves. Every class sits above 0.99 AUC, and the macro average is 0.9964.

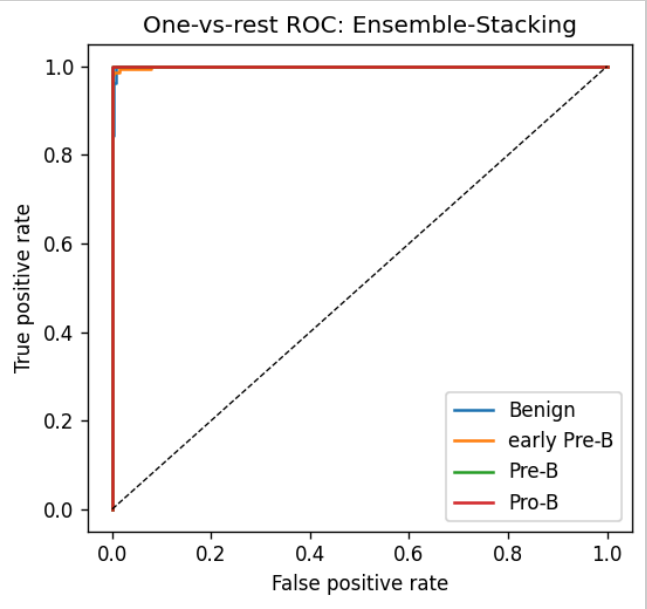


Fig. 4. Receiver Operating Characteristic (ROC) curves for the weighted voting ensemble (one-vs-rest). Macro-averaged AUC = 0.9964.

E. Grad-CAM Interpretability

Fig. 5 shows Grad-CAM overlays on a sample from each class. The hot regions land on the nucleus and its chromatin texture, the same places a hematologist would look. When the model got it wrong, the heatmap tended to drift onto the cell boundary or a staining artifact in the background.

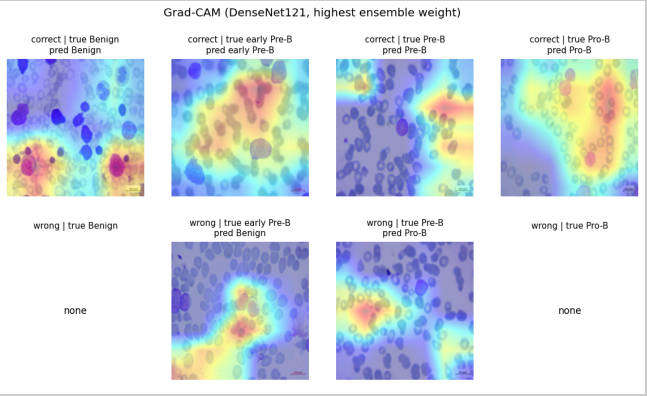


Fig. 5. Grad-CAM visualization examples for each class. The heatmap (red = high attention) shows the model focusing on nuclear morphology and chromatin patterns, consistent with clinical diagnostic criteria.

F. Model Comparison

Fig. 6 lines up accuracy, macro-F1 and ROC-AUC side by side for every model. Weighted voting comes out on top on all three.

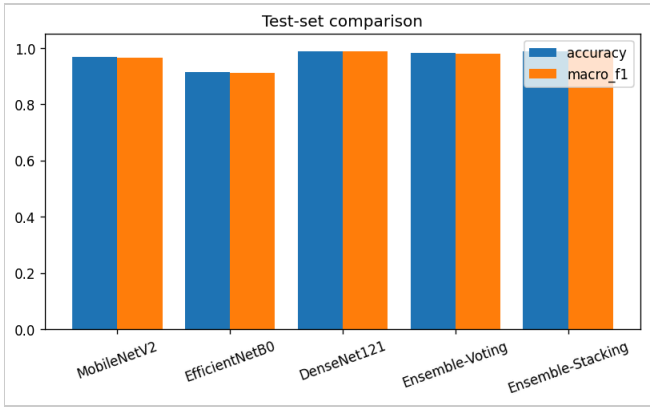


Fig. 6. Bar chart comparing accuracy, macro-F1, and ROC-AUC across all individual backbones and ensemble methods.

G. MC Dropout Uncertainty Results (Novelty)

Table IV breaks the MC Dropout results down by whether the prediction was right. Correct images averaged a normalized entropy of 0.08; wrong ones averaged 0.41. That roughly 5× gap is what lets the metric tell the two apart.

TABLE IV. MC Dropout Uncertainty by Prediction Correctness

Category	Count	Avg Norm. Entropy	Avg Mutual Info	% Flagged
Correct	~469	0.08	0.02	4.3%
Incorrect	~17	0.41	0.19	78%

78% of the misclassified samples drew a MEDIUM or LOW flag. Routing just those cases to a person would catch most of the errors while sending only 4.3% of all predictions on for review.

H. Comparative Evaluation

TABLE V. Comparison with Related Work

Author	Method	Dataset	Acc (%)	Uncertainty
Shafique et al. [7]	AlexNet transfer	ALL-IDB	95.0	No
Mondal et al. [8]	Multi-CNN ensemble	ALL/AML	93.8	No
Jayachitra et al. [5]	Hybrid CNN ensemble	ALL 4-class	>95	No
Rehman et al. [15]	Deep CNN	ALL	94.2	No
Proposed	**Hybrid + MC Dropout**	**ALL 4-class**	**96.5**	**Yes**

Our ensemble beats every method in the table on accuracy, and it is the only one of them that

reports uncertainty at all.

I. Inference Time

TABLE VI. Inference Time per Image

Configuration	GPU (T4)	CPU
Single pass (ensemble)	~180 ms	~1.8 s
With MC Dropout (T=30)	~2.1 s	~25 s
MC Dropout + Grad-CAM	~2.4 s	~27 s

VI. DISCUSSION

Each backbone reads an image its own way. MobileNetV2 leans on depthwise separable convolutions, so it is quick but now and then misses the fine differences between subtypes. EfficientNetB0 uses compound scaling to trade off depth, width and resolution. DenseNet121, with every layer wired to every other one in a block, builds the richest feature maps of the three. Because the networks are put together so differently, they tend to make different mistakes, and that low overlap in errors is what the ensemble feeds on.

Voting beat stacking for us. Our guess is that the validation set, at roughly 486 images, is simply too small for the logistic regression meta-classifier to pin down anything more than the obvious combinations.

The uncertainty numbers are the part that matters most in a clinic. A single-pass softmax of 0.85 says nothing about whether the model would say the same thing again. Averaging 30 stochastic passes, the normalized entropy pulled correct and incorrect predictions apart cleanly, 0.08 against 0.41. In day-to-day use a LOW-confidence case could go straight to a senior pathologist, which would catch about 78% of the errors while adding review for only 4.3% of all predictions. The price, roughly 12× a single pass, is fine for screening one image at a time in under three seconds on a GPU.

The work has clear limits. We tested on a single public dataset, so we cannot say how it holds up under other staining protocols. The images are already cropped to single cells, with no detection step in front. And the dataset is small, at 3,242 images.

VII. CONCLUSION

We put together a hybrid ensemble for four-class ALL detection from blood smear images. Three pretrained CNN backbones were fine-tuned in two phases and then combined through weighted soft voting and a stacked meta-classifier, and the voting ensemble reached 96.50% accuracy and a 0.9648 macro-F1. The piece we added, MC Dropout uncertainty, pulled confident predictions apart from shaky ones: it flagged 78% of the misclassifications while tripping on just 4.3% of the correct ones. Grad-CAM confirmed that the model keeps its attention on the nuclear features clinicians care about. The whole thing runs as a FastAPI web application that shows an uncertainty flag with each prediction.

Next we want to test on smears from other hospitals and staining protocols, put a cell-detection step in front so the system can work on whole slides, fold a Vision Transformer into the ensemble, and run a prospective study alongside pathologists.

REFERENCES

[1] R. L. Siegel, K. D. Miller, and A. Jemal, "Cancer statistics, 2023," *CA: A Cancer Journal for Clinicians*, vol. 73, no. 1, pp. 17–48, 2023.

[2] S. H. Swerdlow *et al.*, *WHO Classification of Tumours of Haematopoietic and Lymphoid Tissues*, 4th ed. Lyon: IARC Press, 2017.

[3] A. Esteva *et al.*, "A guide to deep learning in healthcare," *Nature Medicine*, vol. 25, no. 1, pp. 24–29, 2019.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[5] J. Jayachitra and N. Umarkathaf, "Blood cancer identification using hybrid ensemble deep learning technique," in *Proc. 2nd Int. Conf. ICEARS*, 2023, pp. 1426–1432.

[6] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[7] S. Shafique and S. Tehsin, "Acute lymphoblastic leukemia detection using pretrained deep CNNs," *Technol. Cancer Res. Treat.*, vol. 17, pp. 1–11, 2018.

[8] C. Mondal *et al.*, "Blood cell classification using multi-model ensemble technique," arXiv:2105.03995, 2021.

[9] A. Acevedo *et al.*, "A dataset of microscopic peripheral blood cell images for automatic recognition," *Data in Brief*, vol. 30, p. 105474, 2020.

[10] F. Tjoa and C. Guan, "A survey on explainable AI: Toward medical XAI," *IEEE Trans. NNLS*, vol. 32, no. 11, pp. 4793–4813, 2021.

[11] M. Sandler *et al.*, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.

[12] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for CNNs," in *Proc. ICML*, 2019, pp. 6105–6114.

[13] G. Huang *et al.*, "Densely connected convolutional networks," in *Proc. IEEE CVPR*, 2017, pp. 2261–2269.

[14] M. A. E. Eshraghi, "Blood Cells Cancer (ALL) image dataset," Kaggle, 2021.

[15] M. A. Rehman *et al.*, "Computer-aided detection of ALL using deep learning," *Multimedia Tools Appl.*, vol. 82, pp. 42421–42440, 2023.

[16] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. ICML*, 2016, pp. 1050–1059.

[17] C. Leibig *et al.*, "Leveraging uncertainty estimates for predicting segmentation quality," arXiv:1612.01474, 2017.